

Algorithms Problem Solving

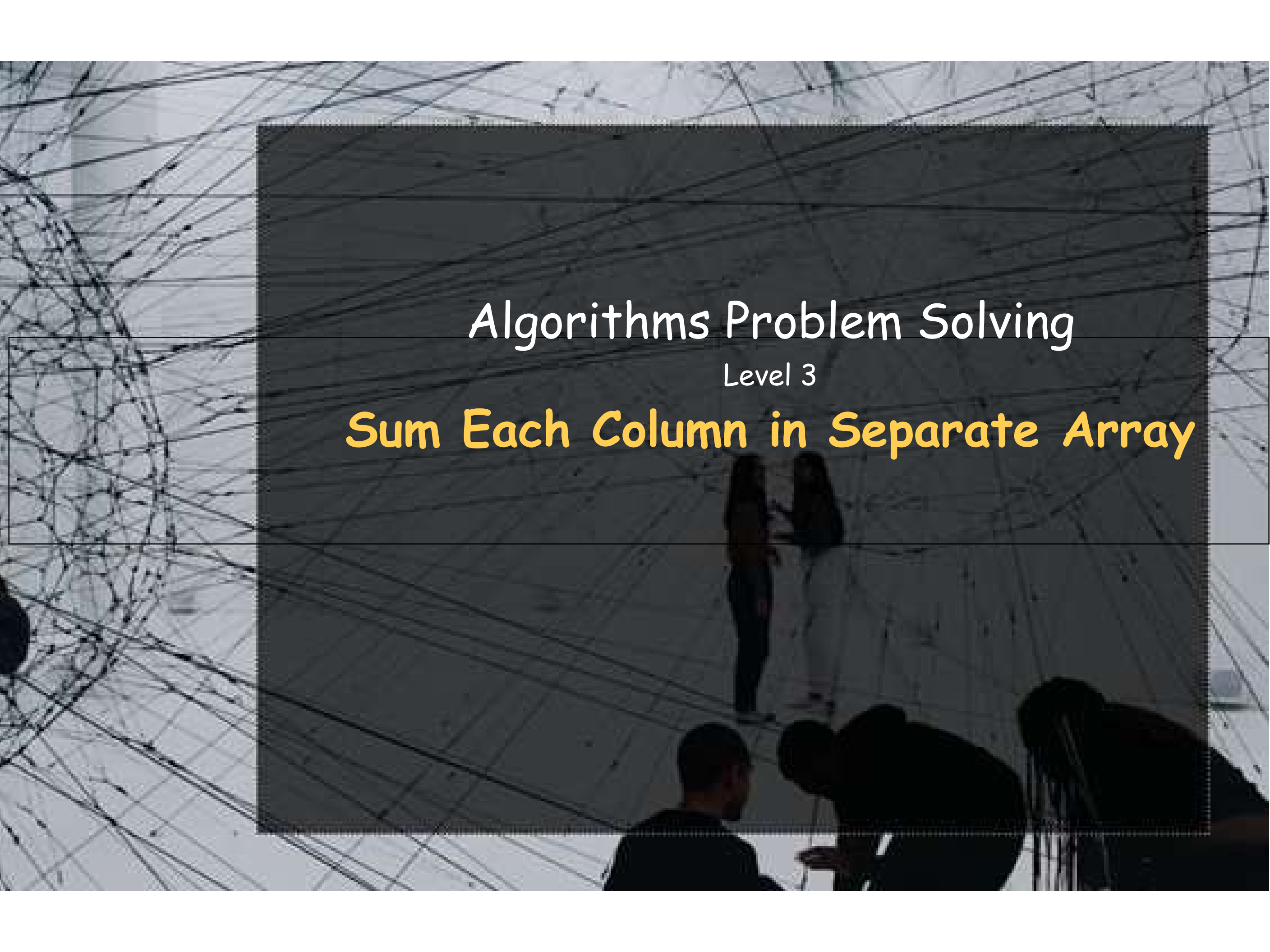
Level 3

Sum Each Column in
Separate Array

Algorithms Problem Solving

Level 3

Sum Each Column in Separate Array



Definition

 Sum Each Column in Separate Array

 Program creates a 3×3 matrix filled with random numbers, calculates the sum of each column, and stores the results in a separate `vector`.

 The original matrix and the column sums are then printed.

Explanation

Columns

 Columns represent elements arranged vertically from top to bottom.

`colSums`

 We calculate the sum of each column by moving through all rows inside each column.

 Each result is stored in the corresponding position of `colSums`.

Storing Results

 Storing the sums in a separate vector makes the results available for later operations, such as calculating column averages.

Procedures

1. Create a 3×3 matrix using `vector<vector<int>>`.
2. Fill the matrix with random numbers between 1 and 100 using `FillMatrixWithRandomNumber()`.
3. Print the matrix using `PrintMatrix()`.
4. Calculate the column sums inside `ColSums()` and store them in a `vector<int>`.
5. Print the column sums using `PrintColSums()`.

💡 Example

The following is a 3x3 random matrix:

5 46 87

1 12 2

87 2 68

➡ Result

The following are the sum of each Col in the matrix:

Col 1 Sum = 93

Col 2 Sum = 60

Col 3 Sum = 157

Note

- ◆ ``srand(time(0))`` makes the random sequence different between program runs.
- ◆ ``vector<int> colSums(columns, 0);`` creates a vector with the same number of elements as the number of columns and initializes them to ``0``.
- ◆ The loop order is important: the outer loop uses column ``j``, while the inner loop uses row ``i``.
- ◆ Each column sum is stored at the corresponding index:
``colSums[j] = sum;``



Comparison

▨ Problem #03 - Sum Each Row in Separate Array

- ✓ Stores row sums in `rowSums`.
- ✓ Each element represents the sum of one row.
- ✓ Calculates the sum horizontally.
- ✓ Uses row index `i` for storage.

▤ Problem #05 - Sum Each Column in Separate Array

- ✓ Stores column sums in `colSums`.
- ✓ Each element represents the sum of one column.
- ✓ Calculates the sum vertically.
- ✓ Uses column index `j` for storage.

✦ Summary

- ✓ Calculate the sum of each column separately.
- ✓ Store the results in a separate `colSums` vector.
- ✓ The outer loop uses column `j`.
- ✓ The inner loop uses row `i`.
- ✓ `colSums[j]` stores the sum of Column `j`.
- ✓ Storing the results separately makes them easier to reuse later.

Thank you 😊